

OAuth 2.0: logowanie przez Google w Expressie

Passport uruchamia strategię Google. Identyfikator klienta i sekret pozostają wyłącznie na backendzie.

1. Gdzie dodać kod w tym projekcie?

Konfigurację GoogleStrategy dodaj do `server/src/config/passport.js`, trasy do `server/src/routes/auth.routes.js`, a dane klienta do lokalnego `server/.env`.

2. Instalacja i konfiguracja strategii

```
npm install passport-google-oauth20

passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: 'http://localhost:3001/api/auth/google/callback',
}, async (accessToken, refreshToken, profile, done) => {
  return done(null, await findOrCreateGoogleUser(profile));
}));
```

3. Trasy Expressa

```
router.get('/google', passport.authenticate('google', {
  scope: ['openid', 'email', 'profile'], state: true,
}));

router.get('/google/callback', passport.authenticate('google', {
  failureRedirect: '/#/access',
}), (req, res) => res.redirect('http://localhost:5173/#/dashboard'));
```

Konfiguracja klienta OAuth w Google

4. Google Cloud Console

Strona konfiguracji: <https://console.cloud.google.com/>

1. Utwórz projekt i skonfiguruj ekran zgody OAuth.
2. **APIs & Services** → **Credentials** → **Create credentials** → **OAuth client ID**.
3. Wybierz **Web application**.
4. Dodaj URI: `http://localhost:3001/api/auth/google/callback`.

←

Utwórz identyfikator klienta OAuth

Typ aplikacji *

Aplikacja internetowa

Nazwa *

Workshop

Nazwa klienta OAuth 2.0. Ta nazwa służy tylko do identyfikacji klienta w konsoli i nie będzie widoczna dla użytkowników.

?

Domeny identyfikatorów URI dodane poniżej zostaną automatycznie dodane do [ekranu zgody OAuth](#) jako [domeny autoryzowane](#).

Autoryzowane źródła JavaScriptu

Do stosowania z żądaniami z przeglądarki

+ Dodaj URI

Autoryzowane identyfikatory URI przekierowania

Do stosowania z żądaniami z serwera WWW

Identyfikator URI 1 *

`http://localhost:3001/api/auth/google/callback`

+ Dodaj URI

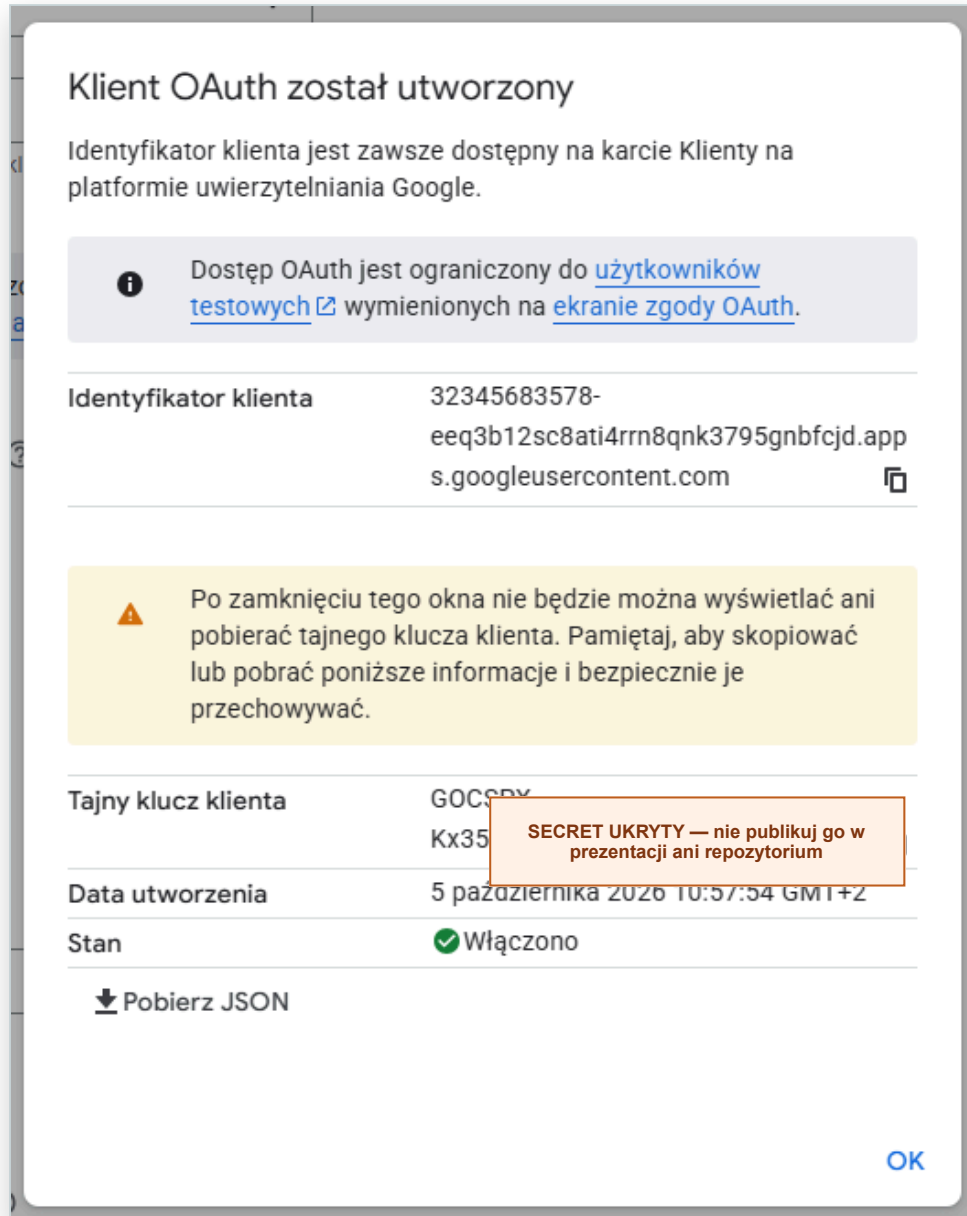
☐ This client will be used by an AI-powered agent

Dokładny adres callbacku wpisujemy w „Autoryzowane identyfikatory URI przekierowania”.

Po utworzeniu klienta OAuth

Skopiuj identyfikator klienta oraz tajny klucz do lokalnego pliku `server/.env`.

```
GOOGLE_CLIENT_ID=...  
GOOGLE_CLIENT_SECRET=...
```



Ekran po utworzeniu klienta OAuth. Identyfikator klienta nie jest sekretem; Client secret jest sekretem.

Ważne: ponieważ sekret był widoczny na dostarczonym zrzucie, zasłoniłem go w PDF. Jeśli został gdziekolwiek opublikowany, wygeneruj nowy sekret w Google Cloud Console.

Ochrona przepływu OAuth

5. Czym jest CSRF?

CSRF (Cross-Site Request Forgery) to sytuacja, w której strona atakującego nakłania przeglądarkę zalogowanej osoby do wysłania żądania do innej aplikacji. Atakujący nie musi odczytać cookie sesyjnego: przeglądarka mogłaby dołączyć je automatycznie.

W OAuth parametr `state` jest losową wartością zapisaną przed przekierowaniem do Google i sprawdzaną po powrocie. Dzięki temu backend rozpoznaje, czy odpowiedź OAuth należy do przepływu rozpoczętego przez tę samą przeglądarkę.

Ważne: nie zapisuj sekretu w Reactcie ani w repozytorium. Konto lokalne wiąż z trwałym `profile.id`, nie tylko z e-mailem.